Andromeda Spec 08: Floating Point

Oct 10, 2026 · @Jerry

Part of the Andromeda spec set. Shared types and conventions are defined in file 00.

Modules: `fpu_top` and its sub-units · **Milestone:** ALU and FPU

1. What this unit does

`fpu_top` executes every RV32F instruction except FLW and FSW (those are loads and stores, handled by the LSU) on port P2. Results follow IEEE 754 single precision, including subnormal numbers, all 5 rounding modes and the 5 exception flags. The FPU is the only unit with long and variable latency besides the LSU, which makes it the main beneficiary of out-of-order execution: independent integer work keeps running while a multiply-add or divide is in progress.

2. How it works

2.1 Sub-units

Sub-unit	Instructions	Target latency	Pipelined
fma	FADD, FSUB, FMUL, FMADD, FMSUB, FNMADD, FNMSUB	4 cycles	Yes
fdivsqrt	FDIV, FSQRT	about 12 to 16 cycles	No; blocks new divides while busy
fcvt	FCVT.W.S, FCVT.WU.S, FCVT.S.W, FCVT.S.WU	2 cycles	Yes
fmisc	FSGNJ/N/X, FMIN, FMAX, FEQ, FLT, FLE, FCLASS, FMV.X.W, FMV.W.X	1 cycle	Yes

Latencies are targets for the first sky130 timing runs, not commitments. The rest of the core doesn't depend on them, because results return through the writeback bus whenever they're ready.

One fused unit for add and multiply. FADD is computed as a × 1 + c and FMUL as a × b + (−0). Using −0 as the addend keeps the sign of a −0 product correct. Sharing one FMA datapath saves a separate adder and multiplier.

2.2 Results in any order

Sub-units have different latencies, so a later FADD can finish before an earlier FDIV. That's fine: every result carries its own `rob_id` and `prd`, and the ROB restores order at commit. All sub-units finish into a 4-entry result buffer that drains one result per cycle onto the P2 writeback bus. `in_ready` drops when the results already in flight could overflow the buffer.

2.3 Rounding and flags

- **Rounding mode.** Each instruction's `rm` field picks the mode. `rm = 111` means dynamic: use `frm` from `csr_unit`. A reserved mode raises an illegal-instruction exception.

- **Exception flags** (invalid, divide by zero, overflow, underflow, inexact) travel with the result in `wb_t.fflags`. The ROB ORs them into the `fflags` CSR only when the instruction commits. Flags from instructions that get flushed never appear, which keeps FP exceptions precise without any extra work in the FPU.
- **NaNs.** Any operation producing a NaN returns the canonical NaN, 0x7FC00000, as RISC-V requires.

2.4 Integer-side instructions

Some FP instructions cross between the files. FEQ, FLT, FLE, FCLASS, FMV.X.W and FCVT.W(U).S write the integer file (`prd.is_fp = 0`). FCVT.S.W(U) and FMV.W.X read their source from the integer file. Rename and register read already route these correctly from the uop's `rd_fp` and `rs1_fp` bits; the FPU just sees 32-bit operands.

3. Interface

Signal	Dir	Width	Description
`flush`	in	1	`be_flush`: drops all in-flight operations, including divides
`in_valid` / `in_uop`	in	1 / uop_t	From rr_stage (op, rm, prd, rob_id)
`in_rs1` / `in_rs2` / `in_rs3`	in	32 / 32 / 32	Operands; rs1 comes from the integer file for int-to-FP ops
`in_ready`	out	1	Can accept this uop (divider and result buffer checked)
`frm`	in	3	Dynamic rounding mode from csr_unit
`wb`	out	wb_t	Result, destination tag and fflags
`wakeup`	out	wakeup_t	Destination tag

4. Implementation notes

4.1 Build or integrate

An IEEE-compliant FPU is a large verification effort on its own: subnormals, rounding of every mode, NaN handling and flags all have to be exactly right. The group should decide early between:

Option	Pros	Cons
Integrate CVFPU (OpenHW Group, formerly FPnew from PULP)	SystemVerilog, open license, configurable formats and pipeline depth, already used in RISC-V cores, includes FMA, conversions and divide/square root	Large codebase to understand; area must be checked under sky130; needs a wrapper to our interface
Integrate Berkeley HardFloat	Very well tested, used in Rocket and BOOM	Written in Chisel, so Verilog must be generated and wrapped

Option	Pros	Cons
Write our own	Full control, best learning experience, can be sized exactly to FP32	Months of work to reach full IEEE compliance

A reasonable path is to integrate an existing FPU behind `fpu_top` for v1 so the core can run FP programs early, and optionally replace sub-units with our own later.

4.2 If writing our own

- **FMA datapath.** Multiply the 24-bit significands (a 24 × 24 multiplier, usually Booth-encoded with a Wallace or Dadda tree), align the addend in parallel, add, count leading zeros, normalize and round. Split it into 4 stages at the natural boundaries: multiply, add, normalize, round.
- **Divide and square root.** An iterative radix-2 or radix-4 SRT unit is small and fits the "blocks while busy" design. The result is rounded by the same rounding logic as the FMA.
- **Subnormals.** Handle them in hardware; RISC-V has no "flush to zero" mode. The leading-zero count in normalization covers most of it.

4.3 Common to both

- **Wrapper responsibilities.** Translate `op` and `rm` into the FPU's own operation codes, carry `rob_id` and `prd` as a tag alongside each operation, and collect results into the result buffer.
- **Flush.** All in-flight operations must be dropped on `be_flush`, including a divide in its middle. With an external FPU, check that it has a kill or flush input; otherwise drop results by tag in the wrapper.
- **Multiplier area.** The 24 × 24 multiplier is the biggest single piece of logic in the FPU. Synthesize it early in sky130.

5. Verification checklist

- ☐ Berkeley TestFloat vectors for every operation and rounding mode, compared result and flags
- ☐ Subnormal inputs and outputs, signed zeros, infinities, quiet and signalling NaNs
- ☐ Canonical NaN on every NaN-producing operation
- ☐ Dynamic rounding mode follows `frm`; reserved modes trap
- ☐ Out-of-order completion: FDIV followed by fast FADDs, results tagged correctly
- ☐ Flush during a divide and during a full FMA pipeline
- ☐ riscv-tests and riscv-arch-test F suites pass on the full core